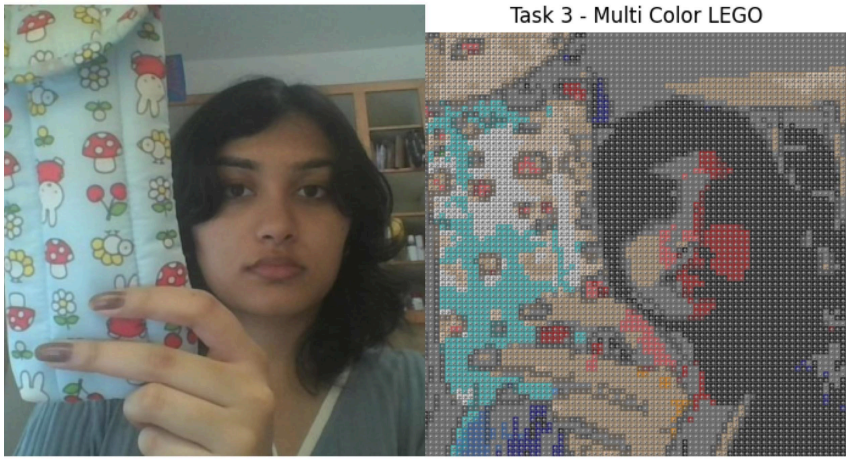

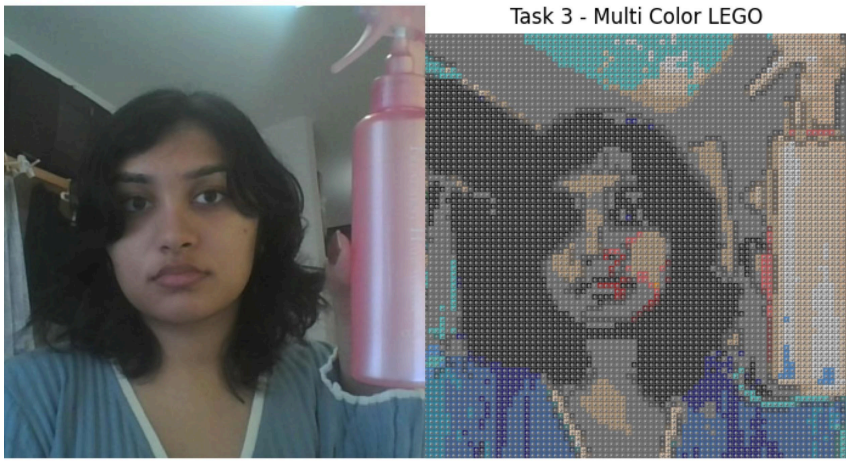

This assignment focuses on capturing real-world images and converting them into LEGO images using image processing techniques. The assignment begins with basic camera capturing and ends with the conversion of the captured images into gray-scale LEGO and later into color LEGO mosaics with varying sizes. The main objectives of the assignment are to capture real-world images as LEGO-style renderings using constrained color palettes, to implement multi-brick merging strategies, to generate brick-usage statistics, and to evaluate robustness in real-world scenarios.

The program was implemented in stages corresponding to the four tasks. In the first task, which includes camera capture, the webcam was accessed using a JavaScript interface in Google Colab, which facilitated the capture of images in real time. The image captured was then converted to RGB format to display it properly. The testing of the code was done by capturing various images of the real world under different lighting conditions.

In Task 2, the image was resized to ensure that the LEGO grid was not larger than 100x100 pixels, converted to grayscale, smoothed with a Gaussian blur to minimize noise, quantized to three colors (black, gray, and white), and then rendered as LEGO bricks of size 1x1. The image was enhanced with edges using the edge detection operator (Canny operator). The edges were superimposed as black bricks to enhance the definition of faces as well as objects. The image produced in Task 2 had to be rendered with only three colors. The effect of

this was that there was considerable loss of detail in the image. This was expected. In Task 2, only 1x1 bricks were used. This was achieved by not applying any brick-merging logic.

In order to accomplish Task 3, the program was expanded to include a manually defined 30-color palette for LEGO bricks, multi-brick merging (1x1, 1x2, 2x2, 2x4), and a summary of the bricks that includes a count of each type. Each pixel will be matched to the closest color in the palette based on Euclidean distance in RGB space. In order to make the color more vibrant, a slight modification to the HSV color space's saturation component will be introduced. A greedy algorithm will be used to combine two adjacent bricks with the same color. A boolean grid will be used to keep track of the regions, ensuring that there is no overlap. The program will then print out a rendered image, a count of the total bricks, and a count for each type of brick.

The program was tested using multiple images captured under different lighting conditions and backgrounds, including an indoor room with objects, a bright, patterned background, hand gestures, and objects of different colors.Robustness was achieved by applying a Gaussian blur to reduce sensor noise, resizing images to keep within computation limits, applying saturation boost to prevent color washing out, and enhancing edges to retain image structure. The system produced recognizable images resembling LEGO bricks in all the test cases.

There were several technical issues encountered.At first, the task 3 results looked almost grayscale. All of these problems arose because of the large number of real-world pixels being mapped to neutral colors in the palette, which was solved by increasing the palette of the LEGO image, increasing the saturation before quantization, and changing the palette values to include stronger blues, reds, and greens. Another problem was the loss of structural detail without edge enhancement facial features were unclear. Edge detection was integrated into both Task 2 and 3 pipelines to preserve contours. There was also excessive noise in the palette mapping, resulting in scattered colors. This was improved by carefully curating the LEGO palette and having a slight smoothing before quantization.

GenAI has been used during the development phase to debug OpenCV-related errors, such as data type-related errors during color conversion; to identify the dominance of grayscale in Task 3; to optimize the approach to color quantization; to optimize the approach to merging bricks; and to optimize the approach to edge detection without compromising the structure of the images. GenAI has been particularly effective in interpreting why the image looks gray despite the presence of multiple colors. GenAI has explained the implications of using distance mapping on the RGB scale and has provided suggestions on adjusting the saturation and improving the palette. GenAI has also been effective in debugging runtime errors, such as the lack of support for the depth of images in OpenCV’s color conversion functions.

GenAI processes the problem by interpreting the image-processing pipeline conceptually, identifying mismatches between expected and actual outputs, explaining the underlying mathematical operation, and suggesting the structured debugging steps. It analyzes both the structure and the visual output, enabling iterative refinement.

Even though GenAI was useful, some limitations were found, such as initial suggestions being too noisy or unrealistic, the need for manual tuning for palette design after receiving AI suggestions, the need for manual tuning of the threshold for edge detection, and the greedy approach for brick merging not being globally optimal. Future improvements could include

K-means clustering in perceptual color space, adaptive edge weighting, and optimized brick tiling using dynamic programming.

Finally, the system meets all the requirements of the assignment, including the real-world camera capture, 3-colored LEGO rendering with 1x1 bricks, multi-colored LEGO rendering with different brick sizes, brick usage summary, and performance with various scenarios. This project demonstrates the trade-offs between color limitation and visual fidelity, as well as the computational challenges of image quantization and brick merging.